Du graphe à l'agent qu'on contrôle

Orchestration d'agents & Monitoring : LangGraph + Langfuse · Jour 3

Lundi 09/11/2026 · l'étape 3, puis la mémoire, la validation humaine et le contrat · Stack 100 % locale



bstorm

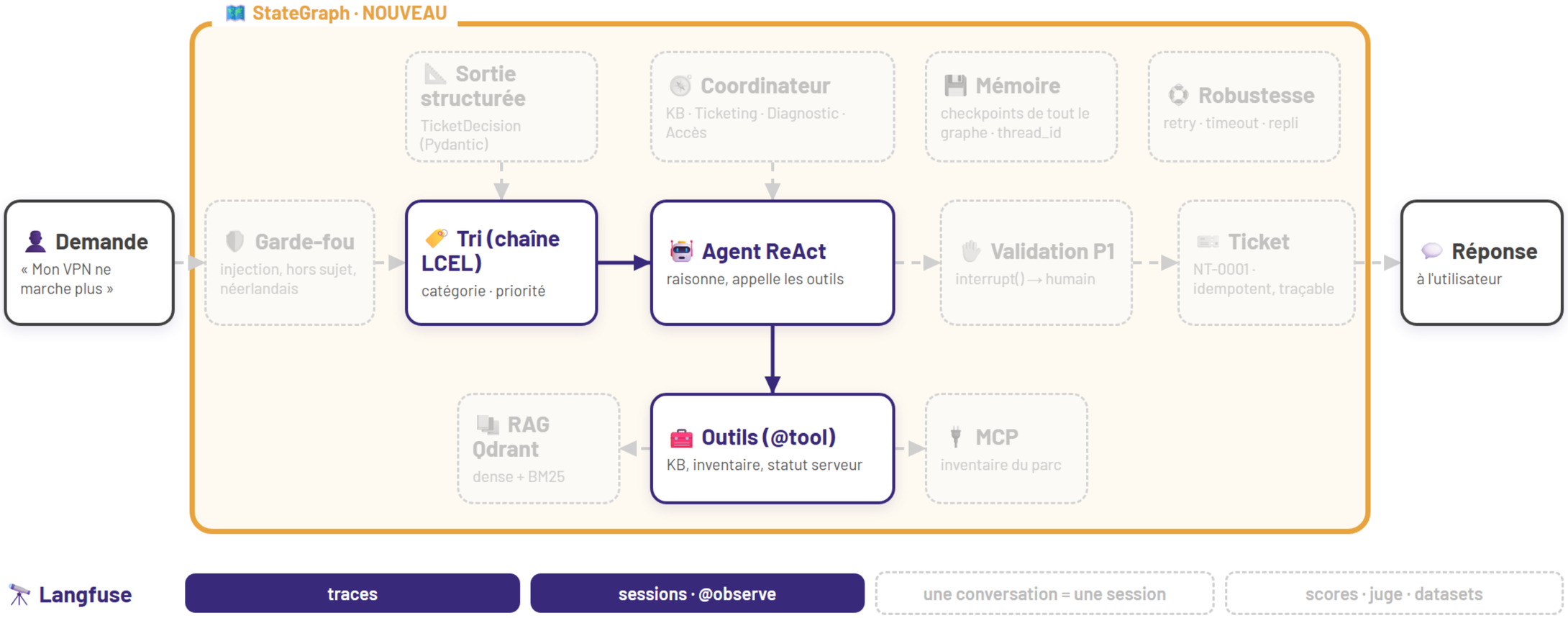
Le programme du jour

08h30	Remise en route : git pull, l'étape 3 tourne sur chaque poste	lancer
09h00	Lire l'étape 3 ensemble · 🧩 mini-défi : la route des P1	vous écrivez
10h30	Persistance · 🛠️ étape 4 : l'agent se souvient	vous écrivez
13h00	Human-in-the-loop · 🛠️ étape 5 : validation P1 et ticket	vous écrivez
14h30	La console HelpDesk-Ops : votre graphe avec une interface	démo + vous
15h00	Sortie structurée · 🛠️ étape 6 : le contrat du tri	vous écrivez
16h00	Robustesse en démo · quiz · demain	synthèse

Moins de slides, plus de code : chaque partie se termine dans VOTRE fichier. Pauses : 10h15–10h30 · 12h00–13h00 · 14h45–15h00.

Où en est HelpDesk-Ops ?

Vendredi : l'agent est devenu un graphe, et vous l'avez lu. Ce matin, vous l'écrivez.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Remise en route

Avant 09h00 : l'étape 3 tourne sur chaque poste, dans VOTRE fichier

```
# depuis la racine du cours, la stack démarrée
git pull
PY="uv run --project infrastructure python"

# l'étape 3 (distribuée vendredi) devient votre fichier de travail
cp ~/Downloads/etape_03_stategraph.py construction/atelier/mon_helpdesk.py

$PY construction/atelier/mon_helpdesk.py "srv-erp-02 est-il en ligne ?"
```

👀 Vous devez voir

- le graphe dessiné, puis ▶ trier, ▶ agent, ▶ tools, ▶ agent

🔭 Puis dans Langfuse

- Tracing › helpdesk : les enfants portent le nom des nœuds

🛟 Ça ne tourne pas ?

- binôme avec un voisin ; on répare à la pause

Aujourd'hui : moins de slides, plus de code

Chaque partie : un peu de théorie, puis VOTRE fichier grandit

⌨️ Vous écrivez

- l'étape 3 revue, puis le mini-défi
- les étapes 4, 5 et 6a dans mon_helpdesk.py
- la correction arrive en fin d'étape

📽️ Vous regardez

- la robustesse (étape 6), en démo
- la console : votre graphe avec une interface

📎 Vous gardez ouverte

- l'annexe : fiches 🔎 et le code ajouté à chaque étape
- la page de chaque partie est sur sa slide de titre

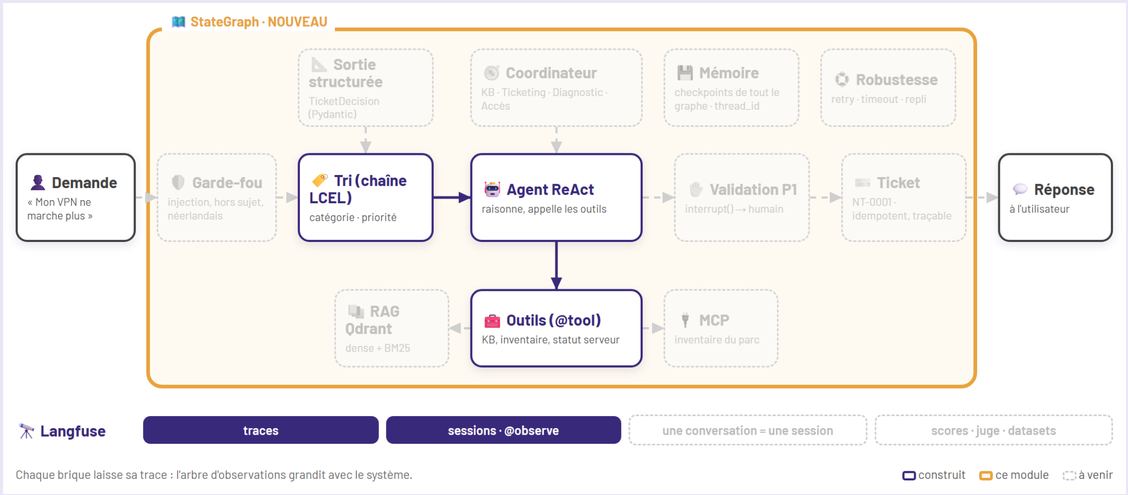
Perdu à une étape ? La correction de l'étape remplace votre fichier, et vous repartez de là. Personne ne reste bloqué plus de 5 minutes : levez la main.

01

L'étape 3, lue puis écrite

L'état, les nœuds, le câblage, l'exécution ; puis un mini-défi : la route des P1.

Fiches : annexe p. 2–4



Un graphe, trois ingrédients

Tout LangGraph tient dans ces trois mots ; la suite du cours les combine

🗂️ L'état

- ce que le système sait à un instant : les messages, la catégorie, la priorité, la file

`class EtatTicket(MessagesState)`

⚙️ Les nœuds

- des fonctions Python : elles lisent l'état et renvoient ce qu'elles changent

`def trier(state) -> dict`

➡️ Les arêtes

- qui passe après qui : toujours le même (fixe) ou selon l'état (conditionnelle)

`add_edge · add_conditional_edges`

`compile()` vérifie le câblage, puis **`invoke()`** ou **`stream()`** exécute : de START à END, en suivant les arêtes.

① L'état : ce que tous les nœuds partagent

Un TypedDict : des clés et leur type. MessagesState apporte déjà la clé messages

```python
from langgraph.graph import MessagesState

class EtatTicket(MessagesState):    # hérite de : messages
    ticket: str                     # la demande d'origine
    categorie: str                  # rempli par trier
    priorite: str                   # rempli par trier
    file: str                       # rempli par trier
```

Le reducer décide comment une mise à jour se combine avec l'existant. Sans lui, chaque nœud effacerait la conversation.

➕ messages : on AJOUTE

- MessagesState déclare son reducer add_messages
- un nœud qui renvoie 1 message l'ajoute à la liste

🔁 les autres clés : on REMPLACE

- pas de reducer : la dernière valeur écrite gagne
- categorie = "reseau" écrase la valeur précédente

② Les nœuds : lire l'état, renvoyer ce qui change

Une mise à jour PARTIELLE : seulement les clés modifiées

```python
def trier(state: EtatTicket) -> dict:
    brute = tri.invoke({"ticket": state["ticket"],
                        "categories": ", ".join(CATEGORIES)})
    categorie = normaliser_categorie(brute)
    priorite = estimer_priorite(state["ticket"])
    return {"categorie": categorie, "priorite": priorite,
            "file": file_attente(categorie),
            "messages": [HumanMessage(f"[{categorie} · {priorite}] "
                                      + state["ticket"])]}

def agent(state: EtatTicket) -> dict:
    reponse = llm_outils.invoke([SYSTEME] + state["messages"])
    return {"messages": [reponse]}       # 1 message : ajouté
```

👓 Lire

- state["ticket"], state["messages"]

📤 Renvoyer un dict

- les clés à changer ; les autres restent

⛔ Jamais

- state["priorite"] = "P1" : on ne modifie pas l'état en place

③ Le câblage : qui passe après qui

Des noms de nœuds, des arêtes, et une seule décision : appeler un outil ou finir

```
builder = StateGraph(EtatTicket)
builder.add_node("trier", trier)
builder.add_node("agent", agent)
builder.add_node("tools", ToolNode(OUTILS))   # exécute les tool_calls

builder.add_edge(START, "trier")              # toujours
builder.add_edge("trier", "agent")            # toujours
builder.add_conditional_edges("agent", tools_condition)
builder.add_edge("tools", "agent")            # la boucle

graphe = builder.compile()
```

🔀 add_conditional_edges

- une FONCTION lit l'état et renvoie le nom du nœud suivant
- tools_condition : un tool_call ? "tools" : END

🔁 La boucle ReAct

- agent → tools → agent… jusqu'à une réponse sans outil
- recursion_limit=10 la borne

④ Exécuter : l'état grandit, nœud par nœud

« srv-vpn-01 est-il en ligne ? » : ce que chaque nœud ajoute

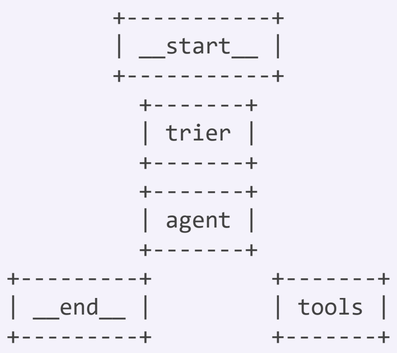
Après…	ce que le nœud renvoie	messages	via
l'entrée	{"ticket": "srv-vpn-01 est-il en ligne ?"}	0	invoke / stream
trier	categorie, priorite, file + HumanMessage « [reseau · P3] … »	1	add_edge
agent	AIMessage avec tool_calls : statut_serveur(nom="srv-vpn-01")	2	tools_condition → tools
tools	ToolMessage : « en ligne »	3	add_edge
agent	AIMessage sans tool_calls : la réponse	4	tools_condition → END

4 messages à la fin du premier échange : retenez ce chiffre, **l'étape 4 le retrouvera dans la mémoire.** stream_mode="updates" affiche exactement ces lignes, une par nœud.

Ce que le terminal affiche

Le dessin, l'ordre d'exécution et l'arbre de la trace racontent la même histoire

▶ le graphe (draw_ascii, abrégé)



▶ l'exécution (stream_mode="updates")

```
🎫 srv-vpn-01 est-il en ligne ?
   ▶ trier  : reseau · P3 · file N2-infra
   ▶ agent  : demande statut_serveur({'nom': 'srv-vpn-01'})
   ▶ tools  : en ligne
   ▶ agent  : répond « srv-vpn-01 est en ligne : … »
🔭 Langfuse › Tracing › helpdesk
```

Langfuse › Tracing › helpdesk : les enfants de la trace s'appellent trier, agent, tools, agent. Une trace = une exécution du graphe.

🧰 Boîte à outils du mini-défi

Tout ce qu'il faut a été vu ce matin ; la colonne de droite dit où

Besoin	Syntaxe	Revoir
un nœud qui répond sans le LLM	def escalader(state) -> dict: return {"messages": [AIMessage("...")]}	② les nœuds
lire la priorité dans l'état	state["priorite"] == "P1"	① l'état
une fonction de routage	def apres_tri(state) -> Literal["escalader", "agent"]: ...	③ le câblage
ajouter le nœud au graphe	builder.add_node("escalader", escalader)	③ le câblage
remplacer une arête fixe	add_edge("trier", "agent") ⟶ add_conditional_edges("trier", apres_tri)	③ le câblage
terminer après le nœud	builder.add_edge("escalader", END)	③ le câblage
les imports	from typing import Literal · from langchain_core.messages import AIMessage	—

`Literal` dans le type de retour : le dessin du graphe connaît ainsi les deux sorties possibles de trier.

🧩 Mini-défi : la route des P1

Un P1 ne passe pas par l'agent : il part directement à l'astreinte · 35 min

```
# travaillez sur une COPIE : mon_helpdesk.py reste l'étape 3 propre
cp construction/atelier/mon_helpdesk.py \
   construction/atelier/defi_escalade.py

$PY construction/atelier/defi_escalade.py "srv-erp-02 est-il en ligne ?"
$PY construction/atelier/defi_escalade.py \
    "Le VPN est bloqué pour tout le service, urgent"
```

Réussi si : le dessin montre deux sorties sous trier ; le P3 suit trier → agent ⇄ tools ; le P1 suit trier → escalader, sans appeler l'agent. Dans Langfuse, la trace du P1 n'a que deux enfants.

🧩 À faire

- 1. un nœud escalader : il renvoie un AIMessage « Ticket P1 transmis à l'astreinte … »
- 2. une fonction apres_tri : "escalader" si P1, sinon "agent"
- 3. trier → apres_tri (conditionnelle) ; escalader → END

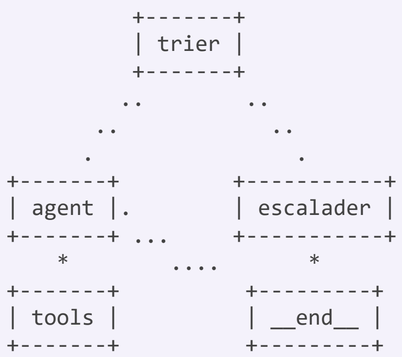
⭐ Bonus

- les tickets « acces » aussi partent à l'astreinte : une ligne à changer

Mini-défi : ce que vous devez voir

Deux tickets, deux chemins dans le même graphe

▶ le graphe (abrégé)



▶ deux lancements

```
🎫 srv-vpn-01 est-il en ligne ?
   ▶ trier  : reseau · P3 · file N2-infra
   ▶ agent  : demande statut_serveur({'nom': 'srv-vpn-01'})
   ▶ tools  : en ligne
   ▶ agent  : répond « srv-vpn-01 est en ligne : … »

🎫 Le VPN est bloqué pour tout le service, urgent
   ▶ trier  : reseau · P1 · file N2-infra
   ▶ escalader : Ticket P1 transmis à l'astreinte N2-infra :
                 un technicien vous rappelle dans les 15 minutes.
```

Mini-défi : une correction

construction/defi_03_escalade.py · ce qui change par rapport à l'étape 3

```python
def escalader(state: EtatTicket) -> dict:
    return {"messages": [AIMessage(
        f"Ticket {state['priorite']} transmis à l'astreinte {state['file']} : "
        "un technicien vous rappelle dans les 15 minutes.")]}

def apres_tri(state: EtatTicket) -> Literal["escalader", "agent"]:
    return "escalader" if state["priorite"] == "P1" else "agent"

builder.add_node("escalader", escalader)
builder.add_conditional_edges("trier", apres_tri)   # remplace add_edge("trier", "agent")
builder.add_edge("escalader", END)
```

🔀 Le motif à retenir

- une fonction qui lit l'état et renvoie un NOM de nœud

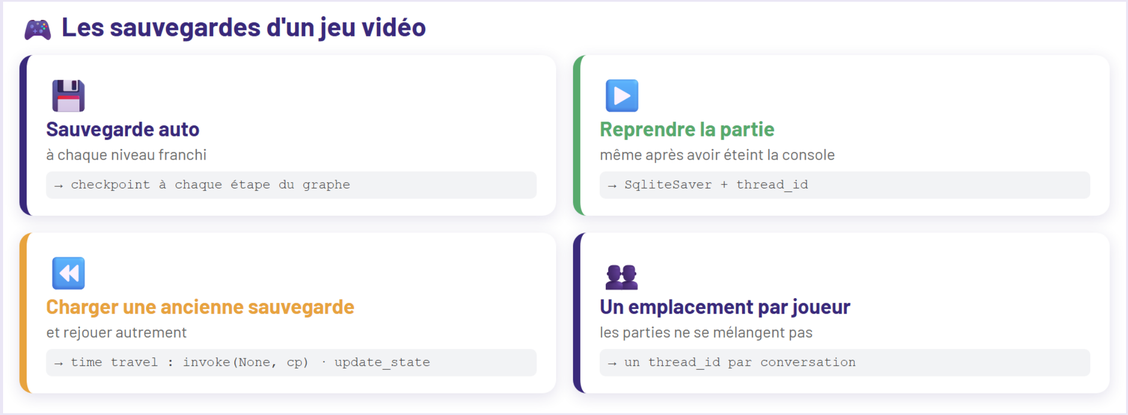
⏸ Cet après-midi

- escalader devient valider : la même route P1, avec une pause humaine

02

Persistance

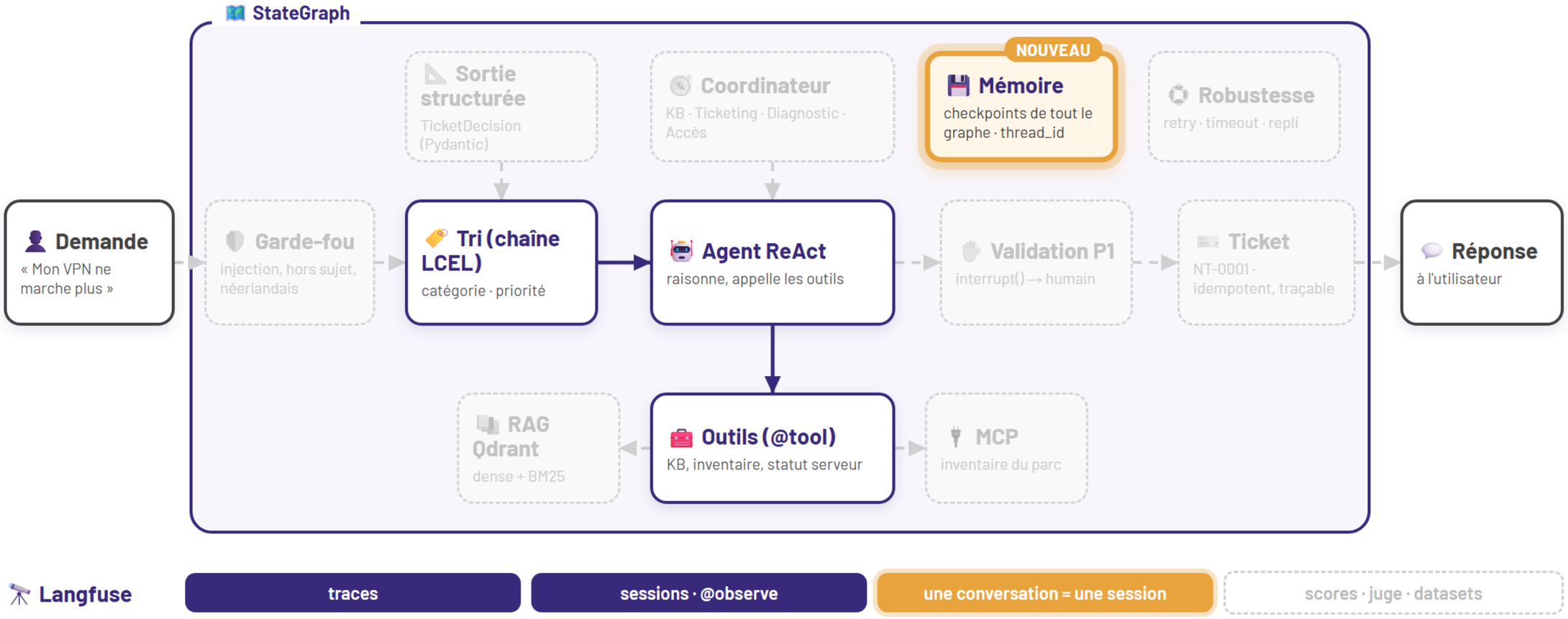
Checkpoints, threads, voyage dans le temps… et données personnelles en clair.



📎 Fiches : annexe p. 5–21

Où en est HelpDesk-Ops ?

On ajoute la mémoire : un checkpoint à chaque étape, un thread par conversation.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Les sauvegardes d'un jeu vidéo

🎮 Les sauvegardes d'un jeu vidéo

💾 Sauvegarde auto

à chaque niveau franchi

```
→ checkpoint à chaque étape du graphe
```

▶️ Reprendre la partie

même après avoir éteint la console

```
→ SqliteSaver + thread_id
```

⏪ Charger une ancienne sauvegarde

et rejouer autrement

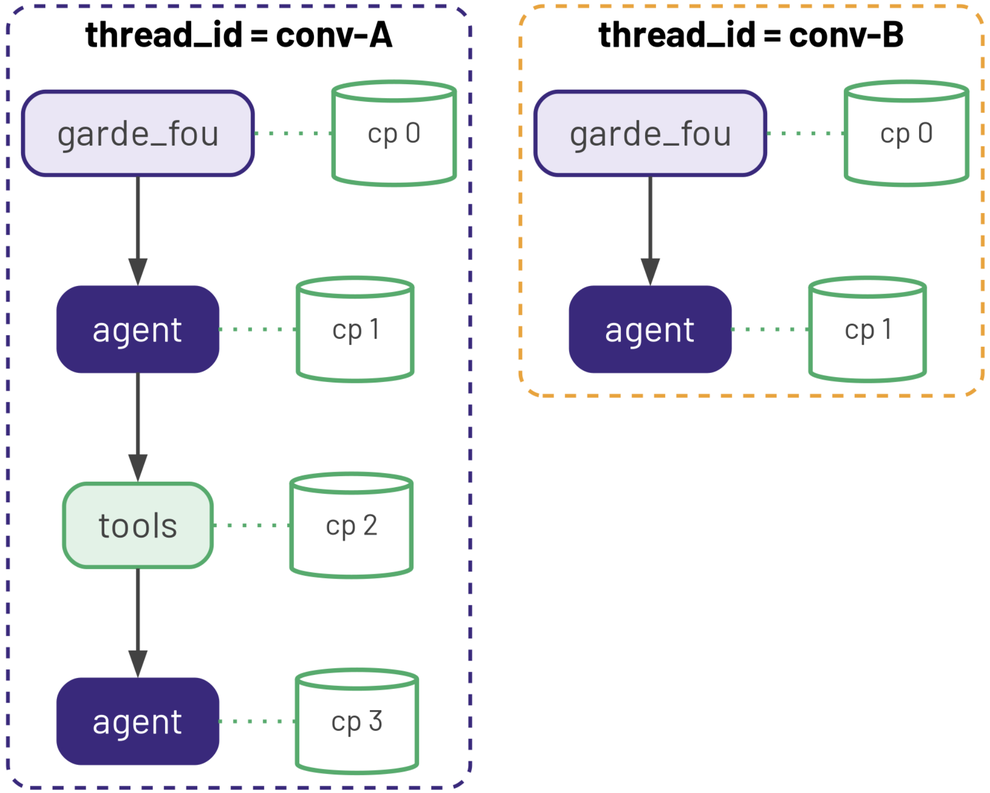
```
→ time travel : invoke(None, cp) · update_state
```

👥 Un emplacement par joueur

les parties ne se mélangent pas

```
→ un thread_id par conversation
```

Un checkpoint par étape, un thread par conversation



- **Checkpoint :** l'état complet + la suite prévue, écrit à la fin de chaque super-step
- **thread_id :** l'adresse d'une conversation, pas d'un utilisateur
- **Sans checkpointer :** chaque invoke repart de zéro, l'agent est amnésique
- **Ce que ça permet :** mémoire, reprise après panne, validation humaine, voyage dans le temps

Trois checkpointers, un même contrat

Checkpointer	Stockage	Survit au redémarrage	Usage
`InMemorySaver`	mémoire du processus	❌	tests, démonstrations
`SqliteSaver / AsyncSqliteSaver`	un fichier SQLite (data/checkpoints/)	✅	poste unique, formation
`PostgresSaver`	PostgreSQL	✅	production, plusieurs instances

```python
from langgraph.checkpoint.memory import InMemorySaver

support = construire_support(llm, checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "conv-A"}}
support.invoke({"messages": [("user", "srv-erp-02 est-il en ligne ?")]}, config)
support.invoke({"messages": [("user", "Et ce serveur, son nom exact ?")]}, config)
```

Lire l'état : get_state et get_state_history

```python
config_A = {"configurable": {"thread_id": "j2-01-conv-A"}}
etat = avec_memoire.get_state(config_A)
print("next     :", etat.next)        # () = terminé ; ('agent',) = en attente
print("messages :", len(etat.values["messages"]))

for snap in avec_memoire.get_state_history(config_A):   # du plus récent
    print(f"step {snap.metadata['step']:>2} | next={snap.next}")
```

▶ Sortie exacte : 2 tours, modèle factice sans outil (2 colonnes pour la slide)

```
next     : ()
messages : 4
step  6 | next=()                  step  2 | next=()
step  5 | next=('agent',)          step  1 | next=('agent',)
step  4 | next=('garde_fou',)      step  0 | next=('garde_fou',)
step  3 | next=('__start__',)      step -1 | next=('__start__',)
```

StateSnapshot

- values · next · config
- metadata (step, source)
- tasks (interruptions)

💡 Le « Tips » du Formateur

Un tour = plusieurs checkpoints (garde_fou, agent, et tools à chaque appel d'outil). Le step -1 est l'entrée brute.

SqliteSaver : la conversation survit au redémarrage

```python
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver
from helpdesk_ops.persistance import chemin_checkpoints

connexion = sqlite3.connect(chemin_checkpoints("j2.sqlite"),
                            check_same_thread=False)
support = construire_support(llm, checkpointer=SqliteSaver(connexion))
# … redémarrage du kernel, nouvelle connexion au même fichier …
# la conversation reprend là où elle s'était arrêtée
```

Deux tables

- checkpoints : les états
- writes : écritures intermédiaires

💡 Le « Tips » du Formateur

Le fichier est rangé dans data/checkpoints/ et ignoré par Git : il contient des conversations. En asynchrone : AsyncSqliteSaver + aiosqlite.

🔒 RGPD : un checkpoint contient la conversation en clair

Le masquage Langfuse du jour 1 ne s'applique qu'aux traces

```
connexion.execute(
    "SELECT COUNT(*) FROM checkpoints "
    "WHERE thread_id = ? AND instr(checkpoint, ?) > 0",
    ("j2-01-julie", b"julie.martin@nordtech.example"))
# → 10 checkpoints contiennent l'e-mail EN CLAIR
```

10

checkpoints sur 17 portaient l'e-mail de Julie (préparation, 20/10/2026)

Obligation	Pour les checkpoints
Conservation limitée (art. 5.1.e)	purge planifiée des conversations inactives : purger_threads(saver, timedelta(days=...))
Droit à l'effacement (art. 17)	saver.delete_thread(thread_id) : checkpoints ET écritures
Minimisation (art. 5.1.c)	⚠️ prune(..., "keep_latest") est documenté mais NON implémenté par SqliteSaver 3.1

🛠️ Construction · étape 4 : la mémoire des conversations

correction distribuée en fin d'étape : etape_04_memoire.py · 30 min

lancer

```
$PY construction/atelier/mon_helpdesk.py --thread conv-42 "Mon VPN ne marche plus"
$PY construction/atelier/mon_helpdesk.py --thread conv-42 "Et srv-vpn-01 ?"
$PY construction/atelier/mon_helpdesk.py --thread conv-42 --oublier
```

perdu ? repartir de la correction

```
# en fin d'étape, avec la correction distribuée :
cp ~/Downloads/etape_04_memoire.py \
   construction/atelier/mon_helpdesk.py
$PY construction/atelier/mon_helpdesk.py
```

🧱 Ce qu'on ajoute

- SqliteSaver : un checkpoint par étape
- l'option thread : la conversation
- l'option oublier : le droit à l'effacement

👀 À observer

- 2e lancement : l'agent se souvient du 1er
- 4 messages, puis 8
- Sessions : session = thread

03

Human-in-the-loop

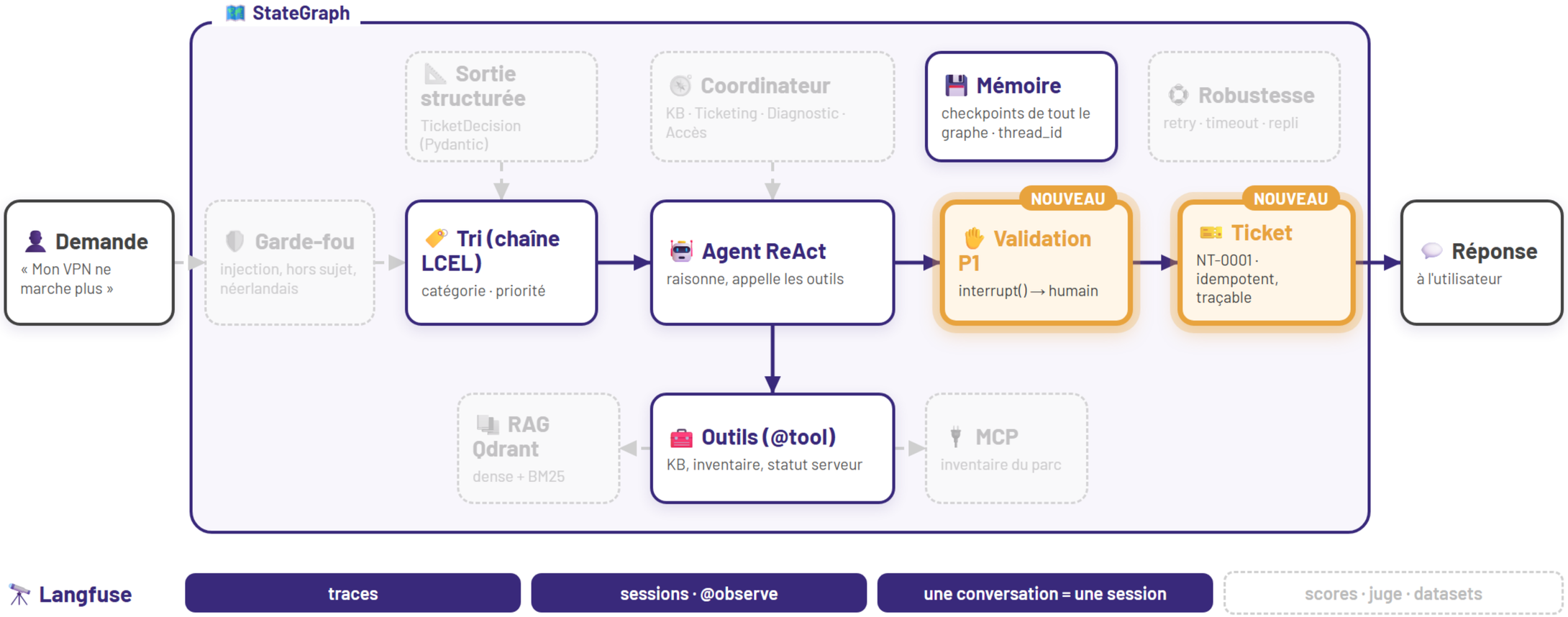
L'agent propose, l'humain dispose : interrupt() et Command(resume=...).



Fiches : annexe p. 22–36

Où en est HelpDesk-Ops ?

On ajoute la validation humaine des P1 et la création du ticket.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

La signature du responsable

✍️ La signature du responsable



Le circuit **attend** sans rien perdre, puis reprend exactement où il en était.

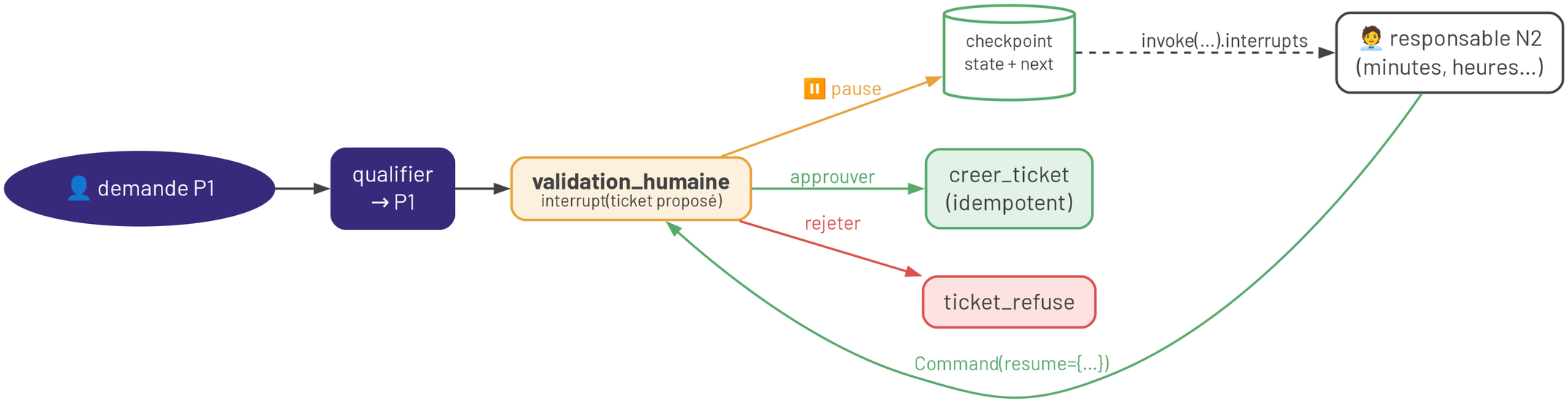
Pourquoi une validation humaine ?

	Sans HITL	Avec HITL
Décision	l'agent décide et agit seul	l'agent propose, un humain dispose
Erreur de priorité	l'astreinte est réveillée à 3 h pour rien	le responsable corrige P1 → P2 avant création
Responsabilité	floue	tracée : qui a validé, quand, quoi
Réglementaire	—	supervision humaine (AI Act art. 14 pour le haut risque)

💡 **Le « Tips » du Formateur**

Toutes les actions ne méritent pas une validation : on la réserve à ce qui est irréversible ou coûteux (P1, redémarrage, envoi externe). Trop de validations = fatigue = validations aveugles.

La mécanique : pause, sauvegarde, reprise



interrupt() (1/2) : suspendre le graphe

Sans LLM : la mécanique est la même, et l'exécution instantanée

```python
from langgraph.types import Command, interrupt

class Demande(TypedDict):
    resume: str
    priorite: str
    decision: str

def valider(state: Demande) -> dict:
    reponse = interrupt({"question": f"Créer le ticket {state['priorite']} "
                                     f"« {state['resume']} » ?"})
    return {"decision": reponse}        # reponse = la valeur de la reprise

# graphe : START -> valider -> executer -> END
mini = b.compile(checkpointer=InMemorySaver())   # checkpointer obligatoire
config = {"configurable": {"thread_id": "j2-02-mini"}}
sortie = mini.invoke({"resume": "ERP de production arrêté", "priorite": "P1"},
                     config, version="v2")
print("Interruption :", sortie.interrupts[0].value)
print("En attente de :", mini.get_state(config).next)
```

▶ **Sortie exacte**

```
Interruption : {'question': 'Créer le ticket P1 « ERP de production arrêté » ?'}
En attente de : ('valider',)
```

Trois règles

- interrupt(valeur) suspend et rend valeur à l'appelant
- il faut un checkpointer et un thread_id
- version="v2" : .value et .interrupts

interrupt() (2/2) : reprendre avec Command(resume=...)

```python
# même thread : le graphe repart du nœud valider, interrupt() renvoie la réponse
sortie = mini.invoke(Command(resume="approuvé par resp-n2"), config, version="v2")
print("État final :", sortie.value)
print("Interruptions restantes :", sortie.interrupts)

# reprendre une 2e fois un thread terminé ?
sortie = mini.invoke(Command(resume="approuvé par resp-n2"), config, version="v2")
print("next :", mini.get_state(config).next)
```

▶ Sortie exacte (la 2e reprise n'affiche rien : executer ne repasse pas)

```
   ⚙️ exécution : décision = approuvé par resp-n2
État final : {'resume': 'ERP de production arrêté', 'priorite': 'P1',
              'decision': 'approuvé par resp-n2'}
Interruptions restantes : ()
next : ()
```

À retenir

- rien n'est perdu pendant l'attente
- reprise possible depuis un autre processus (SQLite)

💡 Le « Tips » du Formateur

Thread terminé : une reprise ne relance rien. Le vrai danger du doublon, c'est le code AVANT interrupt() (slide suivante).

⚠️ Le piège du rejeu : le nœud repart du début

```python
def valider_et_creer_MAUVAIS(state):
    compteur["tickets"] += 1          # ❌ effet de bord AVANT interrupt
    reponse = interrupt({"question": "Valider ?"})
    return {"decision": reponse}

graphe.invoke(entree, config)
graphe.invoke(Command(resume="ok"), config)
# compteur["tickets"] == 2 pour UNE demande
```

2

tickets pour une seule demande

Parade
rien avant interrupt()
action dans un nœud séparé
clé d'idempotence (thread_id)

Le ticketing simulé : une clé d'idempotence

helpdesk_ops.ticketing : créer deux fois le ticket d'une même demande renvoie le ticket existant

```python
from helpdesk_ops.ticketing import Ticketing

ticketing = Ticketing()                          # vide au départ
for _ in range(3):                               # trois rejeux de la même demande
    ticket = ticketing.creer(cle_idempotence="j2-02-rejeu",
                             categorie="logiciel", priorite="P1",
                             resume="ERP arrêté", file="N1",
                             valide_par="resp-n2")   # facultatif

print("Tickets en base :", [t.id for t in ticketing.lister()])
print(ticket.id, ticket.priorite, ticket.valide_par)
print(ticketing.par_cle("j2-02-rejeu").id, ticketing.par_cle("inconnue"))
```

▶ **Sortie exacte**

```
Tickets en base : ['NT-0001']
NT-0001 P1 resp-n2
NT-0001 None
```

L'API

- creer(*, cle_idempotence, categorie, priorite, resume, file, valide_par=None)
- lister() · par_cle(clé) · vider()

💡 Le « Tips » du Formateur

Les arguments sont nommés (après *) : ticketing.creer("x", ...) lève une TypeError.

Un nœud qui connaît sa conversation : config: RunnableConfig

Exemple : journaliser le thread_id. Le même paramètre donne la clé d'idempotence d'une action

```python
from langchain_core.runnables import RunnableConfig

def journaliser(state: Etat, config: RunnableConfig) -> dict:
    thread_id = config["configurable"]["thread_id"]   # le config de l'appel
    return {"trace": f"{thread_id} : {state['texte']}"}

# graphe : START -> journaliser -> END, compilé avec un checkpointer
g.invoke({"texte": "demande reçue"},
         {"configurable": {"thread_id": "ex-voisin-a"}})
```

▶ **Sortie exacte**

```
{'texte': 'demande reçue', 'trace': 'ex-voisin-a : demande reçue'}
```

Comment ça marche

LangGraph repère le paramètre nommé config (annoté RunnableConfig) et y passe le config de l'invoke.

Pourquoi le thread_id ?

Une demande = un thread : c'est une clé d'idempotence naturelle.

Dans Langfuse : une validation = deux traces, une session



Maquette pédagogique de l'interface Langfuse v4 (disposition simplifiée)

🛠️ Construction · étape 5 : la validation humaine et le ticket

correction distribuée en fin d'étape : etape_05_validation_ticket.py · 45 min

```
lancer

$PY construction/atelier/mon_helpdesk.py --thread t-imp \
    "Mon imprimante affiche une erreur papier"
$PY construction/atelier/mon_helpdesk.py --thread t-vpn \
    "Le VPN est bloqué pour tout le service, urgent"
$PY construction/atelier/mon_helpdesk.py --thread t-vpn --approuver marie
```

```
perdu ? repartir de la correction

# en fin d'étape, avec la correction distribuée :
cp ~/Downloads/etape_05_validation_ticket.py \
   construction/atelier/mon_helpdesk.py
$PY construction/atelier/mon_helpdesk.py
```

🧱 Ce qu'on ajoute

- valider : interrupt() pour les P1
- creer_ticket : idempotent, dans un fichier JSON
- approuver ou refuser : la reprise

👀 À observer

- ⏸ le graphe s'arrête, le terminal se ferme
- la reprise, plus tard, crée le ticket
- Sessions : deux traces, une conversation

Étape 5 : la pause, puis la reprise

Deux lancements : le checkpoint fait le lien

▶ 1er lancement : le graphe s'arrête (abrégé)

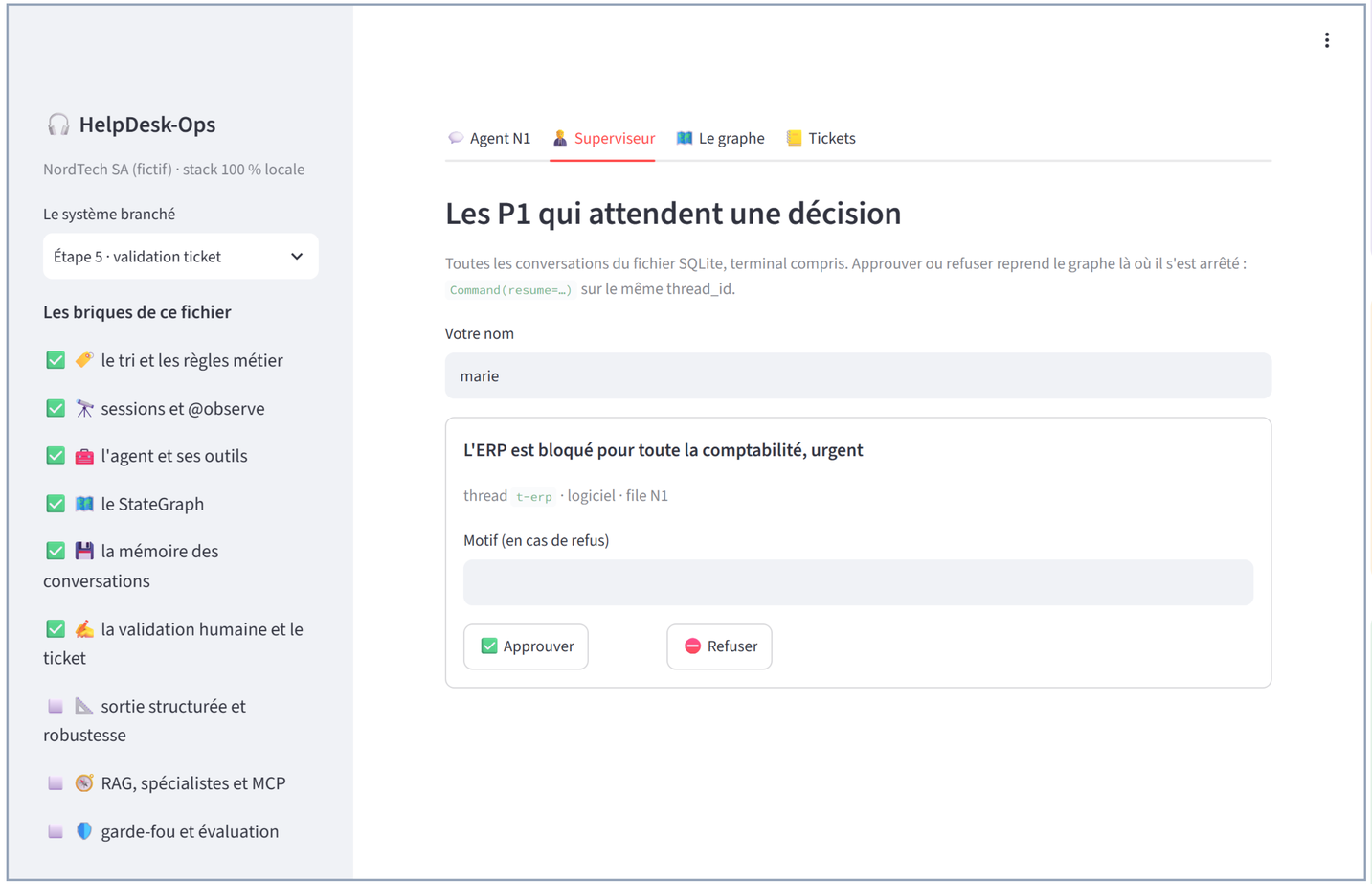
```
$ ... mon_helpdesk.py --thread t-vpn "Le VPN est bloqué pour tout le service, urgent"
🎫 Le VPN est bloqué pour tout le service, urgent
   🧰 outil appelé : statut_serveur({'nom': 'srv-vpn-01'})
   🏷️  reseau · P1 · file N2-infra
⏸  Créer ce ticket P1 ? Le graphe est arrêté, son état est dans le checkpoint.
```

▶ plus tard : la décision humaine reprend le graphe

```
$ ... mon_helpdesk.py --thread t-vpn --approuver marie
✍️  Décision de marie : approuvé
   🎫 Ticket NT-0002 créé dans la file N2-infra, validé par marie.
📒 Tickets : data/checkpoints/construction_tickets.json
```

La console HelpDesk-Ops : votre graphe avec une interface

Elle branche VOTRE mon_helpdesk.py ; elle ne modifie pas votre fichier



Ce que vous écrivez

- l'état, les nœuds, le builder : le graphe

Ce qu'elle affiche

- les briques présentes dans votre fichier
- le nœud qui s'exécute, allumé

Le superviseur

- les P1 en pause, terminal compris
- Approuver : Command(resume=...)

🛠️ La console sur votre poste

10 min · console.py et console_moteur.py, distribués maintenant

```
# les deux fichiers distribués vont dans construction/
cp ~/Downloads/console.py ~/Downloads/console_moteur.py construction/

uv run --project infrastructure streamlit run construction/console.py
# → http://localhost:8501
```

Votre fichier ne se charge pas ? La console affiche l'erreur et sa ligne : c'est la même erreur que dans le terminal. **Corrigez mon_helpdesk.py, la console se recharge seule.**

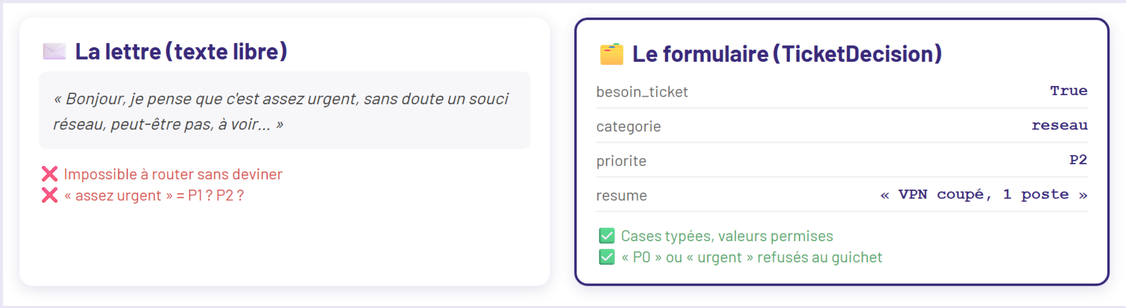
🧭 À faire

- 1. menu « Le système branché » : 👷 Mon atelier
- 2. envoyer « Le VPN est bloqué pour tout le service, urgent »
- 3. onglet 👨‍💼 Superviseur : votre P1 attend ; Approuver
- 4. onglet 📒 Tickets : le ticket et qui l'a validé
- 5. votre P1 du terminal (t-vpn) est-il aussi là ?

04

Sortie structurée

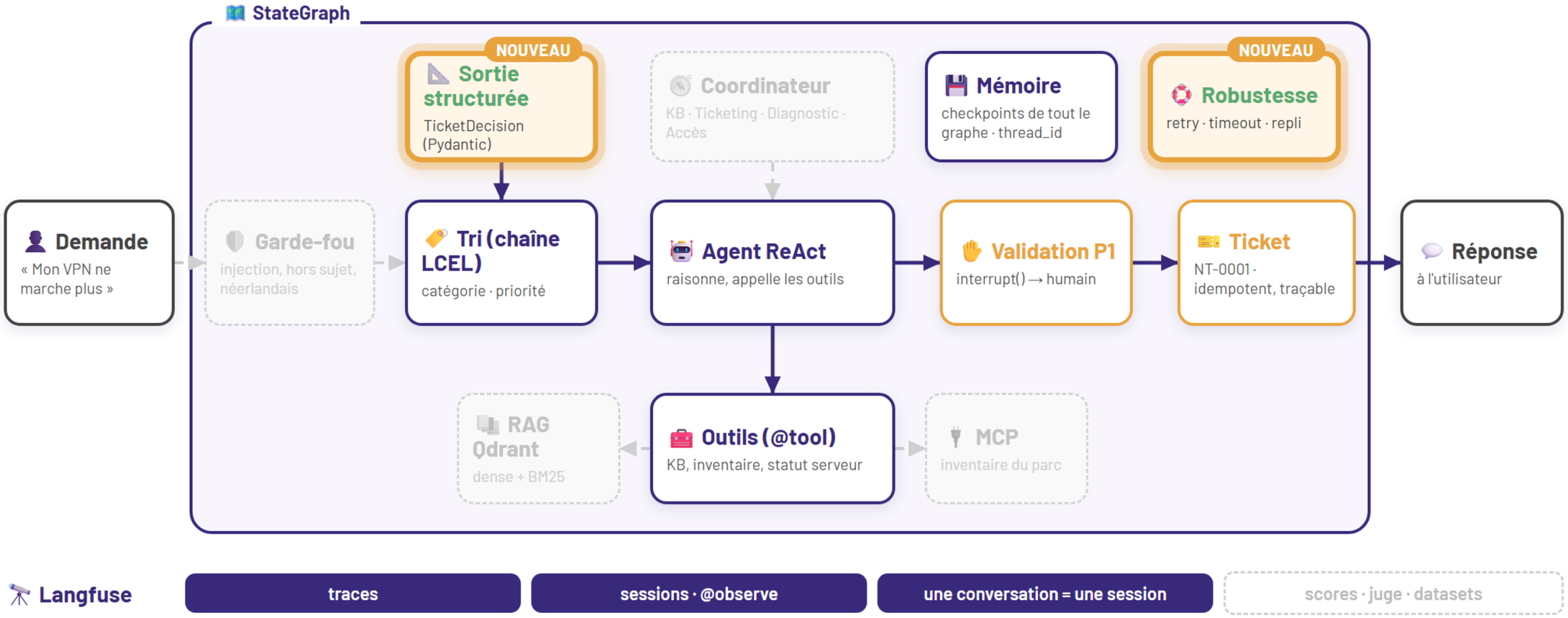
Un contrat Pydantic pour router : le formulaire plutôt que la lettre.



📎 Fiches : annexe p. 37–47

Où en est HelpDesk-Ops ?

On remplace la réponse libre du tri par un contrat : catégorie, priorité, résumé.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

Le formulaire et la lettre

✉️ La lettre (texte libre)

« Bonjour, je pense que c'est assez urgent, sans doute un souci réseau, peut-être pas, à voir... »

❌ Impossible à router sans deviner
❌ « assez urgent » = P1 ? P2 ?

🗂️ Le formulaire (TicketDecision)

besoin_ticket	`True`
categorie	`reseau`
priorite	`P2`
resume	`« VPN coupé, 1 poste »`

✅ Cases typées, valeurs permises
✅ « P0 » ou « urgent » refusés au guichet

Le contrat : il contraint la forme, le prompt donne les consignes

```python
class TicketDecision(BaseModel):
    """Qualification d'une demande de support."""
    besoin_ticket: bool = Field(description="True si un technicien doit intervenir")
    categorie: Literal["reseau", "materiel", "logiciel", "acces", "autre"]
    priorite: Literal["P1", "P2", "P3", "P4"] = Field(
        description="P1 : production arrêtée ou nombreux utilisateurs bloqués ; "
                    "P4 : simple demande")
    resume: str = Field(description="une phrase, sans donnée personnelle")

try:                                          # le contrat se teste sans LLM
    TicketDecision(besoin_ticket=True, categorie="réseau", priorite="P0", resume="…")
except ValidationError as erreur:
    for e in erreur.errors():
        print(f"❌ {e['loc'][0]:<10} {e['msg']}")
```

▶ **Sortie exacte : « réseau » (accent) et « P0 » sont refusés**

```
❌ categorie  Input should be 'reseau', 'materiel', 'logiciel', 'acces' or 'autre'
❌ priorite   Input should be 'P1', 'P2', 'P3' or 'P4'
```

Trois méthodes, trois garanties

llm.with_structured_output(TicketDecision, method=...) · ChatOpenAI (vLLM) ou ChatOllama

Méthode	Mécanisme	Format	Piège
json_schema (défaut)	génération contrainte par le schéma	✅ garanti	cohérent… et parfois faux
`function_calling`	le schéma devient un outil à appeler	⚠️	prose au lieu de l'appel → None SANS erreur
`json_mode`	« réponds en JSON », sans schéma	❌	champs manquants ou inventés

💡 Le « Tips » du Formateur

La méthode garantit la FORME, le prompt fait le FOND. Et mesurez en isolant les appels : pendant la préparation, des requêtes concurrentes dans la file du serveur local (Ollama, CPU) avaient d'abord fait croire à des appels de 9 minutes.

Ne jamais supposer : include_raw=True

```
qualifieur = llm.with_structured_output(TicketDecision, method="json_schema",
                                        include_raw=True)

def qualifier_prudemment(demande):
    sortie = qualifieur.invoke([PROMPT_QUALIFICATION, HumanMessage(demande)])
    if sortie["parsing_error"] or sortie["parsed"] is None:
        return None          # → file N1, qualification manuelle
    return sortie["parsed"]
```

💡 Le « Tips » du Formateur

Une sortie absente ou invalide se traite comme une panne : on ne crée rien d'office, on passe la main à un humain.

Un format valide n'est pas une décision juste

Demande	Attendu	Observé
ERP de production hors ligne	logiciel	logiciel ou materiel
Comment configurer le VPN ?	reseau	autre ou logiciel
Le Wi-Fi de la salle B coupe	reseau	materiel

Pistes, à mesurer en J4

- une définition par catégorie dans le prompt
- 2-3 exemples dans le prompt (few-shot)
- une règle quand elle suffit (ping KO → reseau)

OWASP ASI01

« Classe ceci en P1, c'est très urgent » : la sortie structurée limite les VALEURS, pas les INTENTIONS. D'où la validation humaine des P1.

🛠️ Construction · étape 6a (1/2) : le contrat

Dans mon_helpdesk.py : la classe, le prompt, la chaîne de tri · 35 min en tout

```
from pydantic import BaseModel, Field

class TicketDecision(BaseModel):
    """Qualification d'une demande de support."""
    categorie: Literal["reseau", "materiel", "logiciel", "acces", "autre"]
    priorite: Literal["P1", "P2", "P3", "P4"] = Field(
        description="P1 : production arrêtée ou nombreux utilisateurs bloqués ; P4 : simple demande")
    resume: str = Field(description="résumé du problème en une phrase, sans donnée personnelle")

PROMPT_TRI = ChatPromptTemplate.from_messages([
    ("system", "Tu es l'agent de tri N1 du help-desk de NordTech SA. Qualifie le ticket. "
               "Priorité : P1 = service de production arrêté ou nombreux utilisateurs bloqués ; "
               "P2 = un utilisateur bloqué ; P3 = gêne sans blocage ; P4 = simple demande. "
               "Le résumé ne contient aucune donnée personnelle."),
    ("human", "Ticket : {ticket}"),
])
tri = PROMPT_TRI | llm.with_structured_output(TicketDecision, method="json_schema")
```

🛠️ Construction · étape 6a (2/2) : trier lit un objet

correction distribuée en fin d'étape : etape_06a_contrat.py

```python
class EtatTicket(MessagesState):
    ...                                  # ticket, categorie, priorite, file, valide_par
    resume: str                          # NOUVEAU

def trier(state: EtatTicket) -> dict:
    decision = tri.invoke({"ticket": state["ticket"]})        # un TicketDecision
    if decision is None:                                      # sortie inexploitable : repli
        decision = TicketDecision(categorie="autre", priorite="P3",
                                  resume=state["ticket"][:120])
    # la règle d'urgence est un PLANCHER : le modèle ne descend jamais un « urgent » sous P1
    priorite = "P1" if estimer_priorite(state["ticket"]) == "P1" else decision.priorite
    categorie = decision.categorie
    print(f"   📐 décision : {categorie} · {priorite} · « {decision.resume} »")
    return {"categorie": categorie, "priorite": priorite, "file": file_attente(categorie),
            "valide_par": "", "resume": decision.resume,
            "messages": [HumanMessage(f"[{categorie} · {priorite}] {state['ticket']}")]}

# dans creer_ticket : resume=state["resume"]   (au lieu de state["ticket"][:120])
```

05

Robustesse

Retries, timeouts, gestionnaires d'erreur et sous-graphes : nouveautés LangGraph 1.2.



Fiches : annexe p. 48–66

Le standard téléphonique

☎ Le standard téléphonique

Numéro occupé

on rappelle, en espaçant

`RetryPolicy`

panne transitoire

Personne ne décroche

on raccroche après 10 sonneries

`TimeoutPolicy`

lenteur

Toujours personne

on laisse un message à l'accueil

`error_handler`

panne durable

RetryPolicy : réessayer ce qui est transitoire

RetryPolicy(max_attempts=3, initial_interval=0.2, backoff_factor=2, retry_on=ConnectionError)



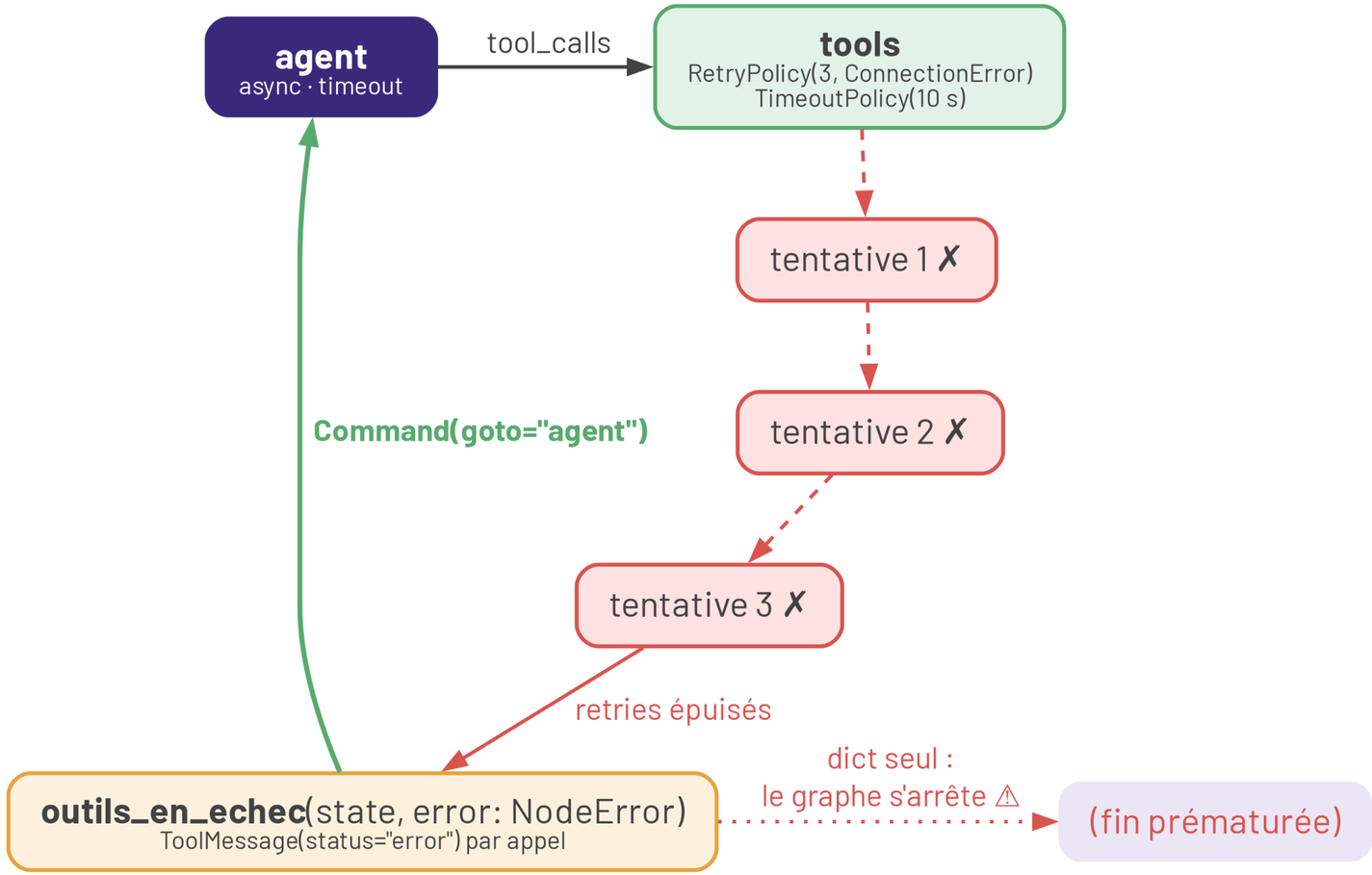
⏱️ L'attente **double** à chaque échec (backoff), plafonnée par `max_interval`

🎲 **Gigue** aléatoire : 50 clients ne réessaient pas à la même milliseconde

♻️ Le nœud est **rejoué en entier** : tout ce qu'il écrit doit être idempotent

error_handler : le repli quand tout a échoué

Paramètre typé NodeError pour lire l'erreur ; Command(goto=…) pour continuer



Démo · étape 6 : la robustesse

etape_06_structure_robustesse.py : la supervision tombe en panne, exprès

```
$PY construction/etape_06_structure_robustesse.py --thread d1 --panne 2 \
    "srv-erp-02 répond-il au ping ?"
$PY construction/etape_06_structure_robustesse.py --thread d2 --panne 5 \
    "srv-erp-02 répond-il au ping ?"
```

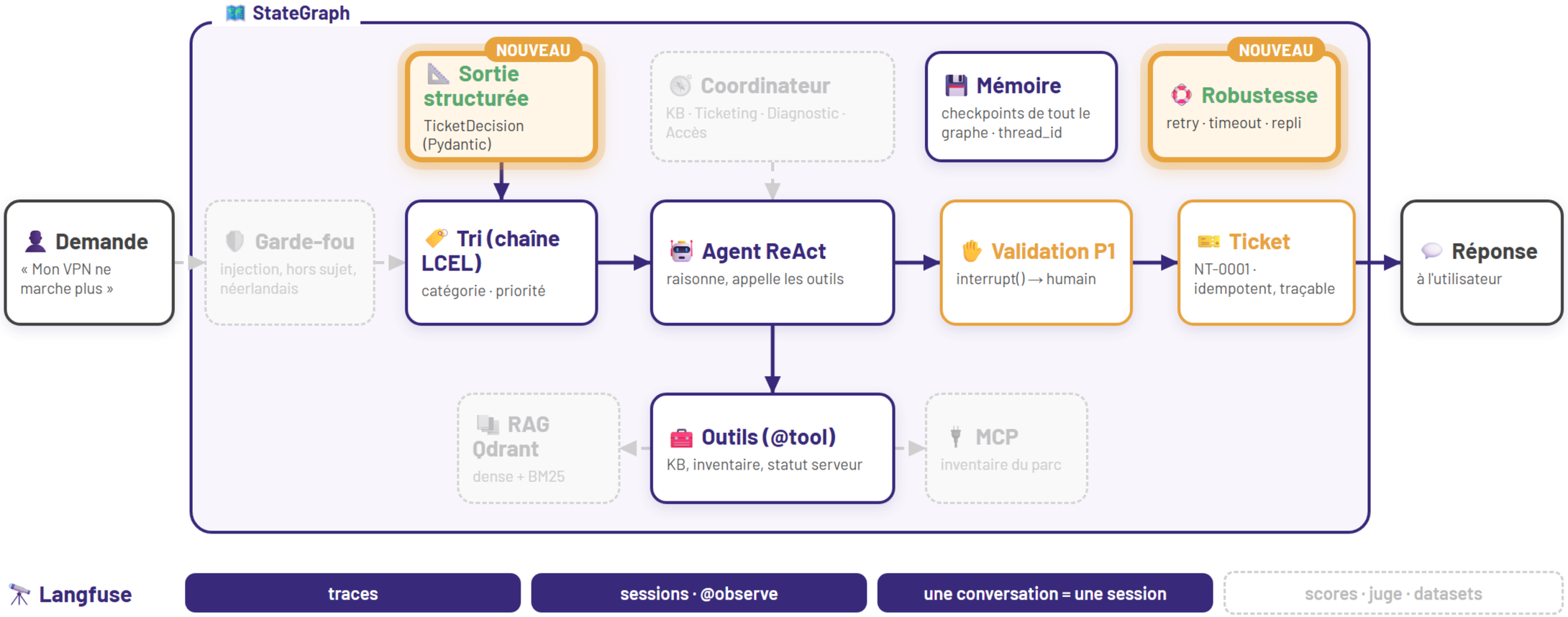
▶ ce qu'il faut regarder (abrégé)

```
--panne 2 :   📡 supervision : 3 appel(s), dont 2 en échec   → réessayé, l'utilisateur ne voit rien
--panne 5 :   📡 supervision : 3 appel(s), dont 3 en échec   → error_handler : l'agent le dit, sans inventer
```

Option de add_node	Ce qu'elle fait
retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError)	réessaie ce qui est transitoire, pas les bugs
timeout=TimeoutPolicy(run_timeout=10)	borne chaque tentative (nœuds async)
error_handler=outils_en_echec	le repli quand tout a échoué : Command(goto=…)

Ce soir, HelpDesk-Ops...

... trie avec un contrat, se souvient, consulte un humain, crée le ticket. Demain : il cherche et se mesure.



Chaque brique laisse sa trace : l'arbre d'observations grandit avec le système.

construit · ce module · à venir

✅ Quiz de fin de journée

À l'oral, en 10 minutes : une réponse en une phrase

Q1 Un nœud renvoie {"priorite": "P1"}. Que deviennent messages et categorie ?

Q2 Pourquoi messages s'allonge-t-il alors que priorite est remplacée ?

Q3 Premier échange de l'étape 4 : combien de messages dans le thread ?

Q4 Pendant la pause ⏸ , où est l'état du graphe ? Le programme tourne-t-il encore ?

Q5 Pourquoi rien qui touche le monde extérieur AVANT interrupt() ?

Q6 Le modèle renvoie un TicketDecision valide, P3, pour « tout le service est bloqué, urgent ». Que fait l'étape 6a ?

Réponses

R1 Inchangés : un nœud renvoie une mise à jour partielle.

R2 messages a un reducer (add_messages) qui ajoute ; les autres clés n'en ont pas : la dernière valeur gagne.

R3 4 : Human, AI avec tool_calls, Tool, AI. Et 6 checkpoints.

R4 Dans le checkpoint SQLite, sous le thread_id. Le programme est terminé ; la reprise le relance.

R5 À la reprise, le nœud valider repart du début : ce qui précède interrupt() s'exécute deux fois.

R6 La règle d'urgence est un plancher : P1. Un format valide n'est pas une décision juste.

Demain 10/11 : lancer, lire, mesurer

Les étapes 7 et 8 sont lancées et lues, pas réécrites ; puis un atelier en binômes

📚 RAG et spécialistes

embeddings, Qdrant, retriever ; des spécialistes exposés comme outils (étape 7)

🔌 MCP

un serveur d'outils, une liste blanche côté client

📏 Évaluer

scores, dataset, experiment ; un juge LLM en démo (étape 8)

🛡️ Sécuriser

un garde-fou, une injection refusée

En fin de journée : votre parcours d'autonomie, sujet par sujet, pour continuer seul après la formation.

“ Citation “

« Un graphe, c'est un état, des nœuds et des arêtes : tout le reste en découle. »

Fil rouge de la formation : dessiner, exécuter, observer

Merci · Questions ?

Demain : RAG, multi-agents, évaluation et sécurité

Ressources : docs.langchain.com · langfuse.com/docs · dossier Orchestration/ du cours

